Pixel Battle — Combat Animation Overhaul Implementation Plan

Pixel Battle — Combat Animation Overhaul Implementation Plan

> **For agentic workers:** REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (`- [ ]`) syntax for tracking.

Goal: Replace the rigid single-line stick-figure renderer with a jointed (elbow + knee) skeleton, 8 per-`vfx`-archetype attack poses, held weapons for the 4 LoL champions, and swing-arc smears — plus a `play_multi` low-action curation filter.

Architecture: Two new modules — `poses.py` (skeleton forward-kinematics + pose tables) and `weapons.py` (weapon registry + drawing) — feed a rewritten `draw_stick_figure` in `stick_renderer.py`. The pose archetype is read from the already-existing `Character.attack_used_kind.vfx`; nothing in `engine/`, `characters.json`, `env.py`, or the RL policy changes. The trained checkpoint stays valid — no retraining.

Tech Stack: Python 3.10, pygame (headless SDL dummy driver), pytest, ffmpeg, stable-baselines3 (only to load the existing checkpoint for the validation render).

Design spec: `docs/superpowers/specs/2026-05-21-pixel-battle-combat-animation-design.md`

File Structure

File	Status	Responsibility
`pixel_battle/rl/poses.py`	**new**	Skeleton FK (`solve_limb`), `FigurePose` / `FigureGeometry` dataclasses, pose interpolation, the 8 archetype pose tables + idle/walk/jump/hit/kick, `select_pose_id` , `compute_figure`
`pixel_battle/rl/weapons.py`	**new**	`Weapon` dataclass, `_WEAPONS` registry by `char.id` , `get_weapon` , `draw_weapon` , `draw_swing_smear`
`pixel_battle/rl/stick_renderer.py`	**modify**	Rewrite `draw_stick_figure` to draw the jointed skeleton + weapons; extend `_STYLES` with split limb lengths; rewrite `_draw_ghost` ; delete `_arm_offsets` / `_leg_offsets` / `_PHASE_DURS` /easing (moved to `poses.py`). Keep `spawn_impact_burst` , `spawn_landing_dust` , `ProjectileLayer` unchanged.

File	Status	Responsibility
`pixel_battle/rl/play.py`	**modify (curation only)**	`run_one_match` counts `hit` events and returns `action_score` (hits/sec) in its result dict
`pixel_battle/rl/play_multi.py`	**modify (curation only)**	Accept a match only if KO **and** `action_score >= MIN_ACTION_RATE`
`pixel_battle/tests/test_skeleton_fk.py`	**new**	FK + clamp tests
`pixel_battle/tests/test_poses.py`	**new**	Pose model, interpolation, selection, distinctness, visual-safety
`pixel_battle/tests/test_weapons.py`	**new**	Weapon registry + drawing
`pixel_battle/tests/test_stick_renderer_pose.py`	**rewrite**	Old `_arm_offsets` / `_leg_offsets` tests are removed (those functions no longer exist); keep the `ProjectileLayer` test; add a jointed-`draw_stick_figure` smoke test
`pixel_battle/tests/test_curation.py`	**new**	`action_score` + `play_multi` filter

Angle convention (used everywhere in `poses.py`): screen space, `+x` right, `+y` **down**, degrees. `0` points right, `90` down, `180` left, `270` up. A limb is two segments; segment 1's angle is absolute, segment 2's angle is a *flex* relative to segment 1. Poses are authored for `facing = +1` (facing right); `compute_figure` mirrors every absolute angle (`d -> 180 - d`) and every flex (`f -> -f`) for `facing = -1` .

Task 1: Skeleton forward-kinematics

Files: - Create: `pixel_battle/rl/poses.py` - Test: `pixel_battle/tests/test_skeleton_fk.py`

- [ ] **Step 1: Write the failing test**

```
# pixel_battle/tests/test_skeleton_fk.py
"""Forward-kinematics for the 2-segment skeleton."""
import math

from pixel_battle.rl.poses import solve_limb, clamp_elbow, clamp_knee


def _dist(a, b):
    return math.hypot(a[0] - b[0], a[1] - b[1])


def test_straight_limb_reaches_full_length():
    root = (0.0, 0.0)
    joint, end = solve_limb(root, seg1_deg=90, flex_deg=0, len1=10, len2=10)
    assert abs(_dist(root, joint) - 10) < 1e-6
    assert abs(_dist(root, end) - 20) < 1e-6
```

```
def test_bent_limb_end_is_closer_than_straight():
    root = (0.0, 0.0)
    _, straight = solve_limb(root, 90, 0, 10, 10)
    _, bent = solve_limb(root, 90, 90, 10, 10)
    assert _dist(root, bent) < _dist(root, straight)


def test_joint_sits_at_seg1_length_regardless_of_flex():
    root = (0.0, 0.0)
    j1, _ = solve_limb(root, 90, 0, 12, 8)
    j2, _ = solve_limb(root, 90, 120, 12, 8)
    assert abs(_dist(root, j1) - 12) < 1e-6
    assert abs(_dist(root, j2) - 12) < 1e-6


def test_clamps_bound_flex():
    assert clamp_elbow(999) == 165.0
    assert clamp_elbow(-999) == -165.0
    assert clamp_knee(999) == 165.0
    assert clamp_knee(-999) == -165.0
    assert clamp_elbow(40) == 40.0
```

- [ ] **Step 2: Run the test to verify it fails**

Run: `python -m pytest pixel_battle/tests/test_skeleton_fk.py -v` Expected: FAIL — `ModuleNotFoundError: No module named 'pixel_battle.rl.poses'`

- [ ] **Step 3: Create `poses.py` with the FK core**

```
# pixel_battle/rl/poses.py
"""Two-segment skeleton + pose tables for the stick-figure renderer.

Angle convention (screen space, +x right, +y DOWN, degrees):
  0 = right, 90 = down, 180 = left, 270 = up.
A limb has two segments. Segment 1's angle is absolute; segment 2's angle
is a flex relative to segment 1. Poses are authored for facing = +1
(facing right); compute_figure() mirrors absolute angles (d -> 180 - d)
and flexes (f -> -f) for facing = -1.
"""
from __future__ import annotations
import math
from dataclasses import dataclass
from typing import Dict, Optional, Tuple

Vec = Tuple[float, float]

# Joint flex clamps (degrees) — guard against degenerate/hyperextended limbs.
```

```
ELBOW_FLEX_MIN, ELBOW_FLEX_MAX = -165.0, 165.0
KNEE_FLEX_MIN, KNEE_FLEX_MAX = -165.0, 165.0


def _deg2vec(deg: float) -> Vec:
    r = math.radians(deg)
    return math.cos(r), math.sin(r)


def solve_limb(root: Vec, seg1_deg: float, flex_deg: float,
               len1: float, len2: float) -> Tuple[Vec, Vec]:
    """Return (joint, end) world positions for a 2-segment limb.

    root     -- proximal anchor (shoulder or hip)
    seg1_deg -- absolute angle of segment 1 (upper arm / thigh)
    flex_deg -- angle of segment 2 relative to segment 1
    """
    c1, s1 = _deg2vec(seg1_deg)
    joint = (root[0] + c1 * len1, root[1] + s1 * len1)
    c2, s2 = _deg2vec(seg1_deg + flex_deg)
    end = (joint[0] + c2 * len2, joint[1] + s2 * len2)
    return joint, end


def clamp_elbow(flex_deg: float) -> float:
    return max(ELBOW_FLEX_MIN, min(ELBOW_FLEX_MAX, flex_deg))


def clamp_knee(flex_deg: float) -> float:
    return max(KNEE_FLEX_MIN, min(KNEE_FLEX_MAX, flex_deg))
```

- [ ] **Step 4: Run the test to verify it passes**

Run: `python -m pytest pixel_battle/tests/test_skeleton_fk.py -v` Expected: PASS (4/4)

- [ ] **Step 5: Commit**

```
git add pixel_battle/rl/poses.py pixel_battle/tests/test_skeleton_fk.py
git commit -m "feat(pixel-battle/rl): 2-segment skeleton forward-kinematics"
```

Task 2: Pose model, easing, and interpolation

Files: - Modify: `pixel_battle/rl/poses.py` - Test: `pixel_battle/tests/test_poses.py`

- [ ] **Step 1: Write the failing test**

```python
# pixel_battle/tests/test_poses.py
"""Pose model, interpolation, selection, distinctness, visual safety."""
import os
os.environ.setdefault("SDL_VIDEODRIVER", "dummy")

from pixel_battle.rl.poses import (
    FigurePose, lerp_pose, ease_in_cubic, ease_out_cubic, ease_in_out_cubic,
)


def _pose(v):
    return FigurePose(torso_lean=v, front_arm=(v, v), back_arm=(v, v),
                      front_leg=(v, v), back_leg=(v, v), weapon_deg=v)


def test_lerp_pose_endpoints():
    a, b = _pose(0.0), _pose(100.0)
    assert lerp_pose(a, b, 0.0).torso_lean == 0.0
    assert lerp_pose(a, b, 1.0).torso_lean == 100.0


def test_lerp_pose_midpoint():
    mid = lerp_pose(_pose(0.0), _pose(100.0), 0.5)
    assert mid.torso_lean == 50.0
    assert mid.front_arm == (50.0, 50.0)
    assert mid.weapon_deg == 50.0


def test_lerp_pose_clamps_t():
    a, b = _pose(0.0), _pose(100.0)
    assert lerp_pose(a, b, -5.0).torso_lean == 0.0
    assert lerp_pose(a, b, 9.0).torso_lean == 100.0


def test_easing_monotonic_0_to_1():
    for ease in (ease_in_cubic, ease_out_cubic, ease_in_out_cubic):
        assert abs(ease(0.0)) < 1e-9
        assert abs(ease(1.0) - 1.0) < 1e-9
        assert 0.0 <= ease(0.5) <= 1.0
```

- [ ] **Step 2: Run the test to verify it fails**

Run: `python -m pytest pixel_battle/tests/test_poses.py -v` Expected: FAIL — `ImportError: cannot import name 'FigurePose'`

- [ ] **Step 3: Append the pose model + easing to `poses.py`**

```python
# --- Append to pixel_battle/rl/poses.py ---

@dataclass
class FigurePose:
    """All joint angles for one figure at one instant (authored facing +1).

    Arm tuples are (shoulder_deg, elbow_flex_deg).
    Leg tuples are (hip_deg, knee_flex_deg).
    """
    torso_lean: float                # deg; + leans toward facing
    front_arm: Tuple[float, float]
    back_arm: Tuple[float, float]
    front_leg: Tuple[float, float]
    back_leg: Tuple[float, float]
    weapon_deg: float                # absolute weapon angle


def ease_in_cubic(t: float) -> float:
    t = max(0.0, min(1.0, t))
    return t * t * t


def ease_out_cubic(t: float) -> float:
    t = max(0.0, min(1.0, t))
    return 1.0 - (1.0 - t) ** 3


def ease_in_out_cubic(t: float) -> float:
    t = max(0.0, min(1.0, t))
    if t < 0.5:
        return 4.0 * t * t * t
    return 1.0 - (-2.0 * t + 2.0) ** 3 / 2.0


def _lerp(a: float, b: float, t: float) -> float:
    return a + (b - a) * t


def _lerp_pair(a: Tuple[float, float], b: Tuple[float, float],
               t: float) -> Tuple[float, float]:
    return (_lerp(a[0], b[0], t), _lerp(a[1], b[1], t))


def lerp_pose(a: FigurePose, b: FigurePose, t: float) -> FigurePose:
    t = max(0.0, min(1.0, t))
    return FigurePose(
        torso_lean=_lerp(a.torso_lean, b.torso_lean, t),
```

```
        front_arm=_lerp_pair(a.front_arm, b.front_arm, t),
        back_arm=_lerp_pair(a.back_arm, b.back_arm, t),
        front_leg=_lerp_pair(a.front_leg, b.front_leg, t),
        back_leg=_lerp_pair(a.back_leg, b.back_leg, t),
        weapon_deg=_lerp(a.weapon_deg, b.weapon_deg, t),
    )
```

- [ ] **Step 4: Run the test to verify it passes**

Run: `python -m pytest pixel_battle/tests/test_poses.py -v` Expected: PASS (4/4)

- [ ] **Step 5: Commit**

```
git add pixel_battle/rl/poses.py pixel_battle/tests/test_poses.py
git commit -m "feat(pixel-battle/rl): FigurePose model + easing + interpolation"
```

Task 3: Pose selection

Files: - Modify: `pixel_battle/rl/poses.py` - Test: `pixel_battle/tests/test_poses.py`

- [ ] **Step 1: Add the failing test**

Append to `pixel_battle/tests/test_poses.py`:

```
from pixel_battle.engine.character import Character
from pixel_battle.rl.poses import select_pose_id


def _char(state="idle", on_ground=True, vel_x=0.0):
    c = Character.load("garen")
    c.action_state = state
    c.on_ground = on_ground
    c.vel_x = vel_x
    return c


def test_select_idle_walk_jump_hit():
    assert select_pose_id(_char("idle")) == "idle"
    assert select_pose_id(_char("idle", vel_x=2.0)) == "walk"
    assert select_pose_id(_char("idle", on_ground=False)) == "jump"
    assert select_pose_id(_char("hit_stagger")) == "hit"


def test_select_attack_uses_vfx_archetype():
    c = _char("attacking")
    c.attack_anim_hint = "jab"
    c.attack_used_kind = c.skills[2]           # garen judgment, vfx "spin"
```

```
    assert c.attack_used_kind.vfx == "spin"
    assert select_pose_id(c) == "spin"


def test_select_attack_unknown_vfx_falls_back_to_melee():
    c = _char("attacking")
    c.attack_anim_hint = "jab"

    class _Fake:
        vfx = "nonsense"
    c.attack_used_kind = _Fake()
    assert select_pose_id(c) == "melee"


def test_select_kick_overrides_archetype():
    c = _char("attacking")
    c.attack_anim_hint = "kick"
    c.attack_used_kind = c.skills[0]            # basic, vfx "melee"
    assert select_pose_id(c) == "kick"
```

- [ ] **Step 2: Run the test to verify it fails**

Run: `python -m pytest pixel_battle/tests/test_poses.py::test_select_attack_uses_vfx_archetype -v`
Expected: FAIL — `ImportError: cannot import name 'select_pose_id'`

- [ ] **Step 3: Append `select_pose_id` to `poses.py`**

```
# --- Append to pixel_battle/rl/poses.py ---

ARCHETYPE_IDS = frozenset(
    {"melee", "slam", "spin", "dash", "bolt", "multishot", "aura", "beam"})


def select_pose_id(char) -> str:
    """Pick the pose key for `char`'s current state.

    Attacking: `kick` if tagged so, else the skill's vfx archetype
    (unknown -> `melee`). Otherwise idle / walk / jump / hit.
    """
    if char.action_state == "attacking":
        if getattr(char, "attack_anim_hint", "") == "kick":
            return "kick"
        skill = getattr(char, "attack_used_kind", None)
        vfx = getattr(skill, "vfx", "melee") if skill is not None else "melee"
        return vfx if vfx in ARCHETYPE_IDS else "melee"
    if not char.on_ground:
        return "jump"
    if char.action_state == "hit_stagger":
```

```
        return "hit"
    if abs(char.vel_x) > 0.5:
        return "walk"
    return "idle"
```

- [ ] **Step 4: Run the tests to verify they pass**

Run: `python -m pytest pixel_battle/tests/test_poses.py -v` Expected: PASS (all)

- [ ] **Step 5: Commit**

```
git add pixel_battle/rl/poses.py pixel_battle/tests/test_poses.py
git commit -m "feat(pixel-battle/rl): pose selection by vfx archetype"
```

Task 4: Non-attack poses + `compute_figure`

Files: - Modify: `pixel_battle/rl/poses.py` - Test: `pixel_battle/tests/test_poses.py`

This task adds the static poses (idle/walk/jump/hit), the `FigureGeometry` output type, and `compute_figure` — which resolves a pose, runs FK, and positions the figure so the **lower of the two feet sits exactly at `char.pos_y`** (feet planted by construction).

- [ ] **Step 1: Add the failing test**

Append to `pixel_battle/tests/test_poses.py`:

```
from pixel_battle.rl.poses import compute_figure, FigureGeometry

# Minimal style for tests (matches the _STYLES schema from Task 8).
_TEST_STYLE = {
    "head_shape": "circle", "head_size": 26, "torso_length": 80,
    "upper_arm": 30, "forearm": 30, "thigh": 34, "shin": 34,
    "line_width": 7, "hand_radius": 6, "foot_length": 18,
}


def _standing(char_id="garen", facing=1):
    c = Character.load(char_id)
    c.pos_x, c.pos_y = 240.0, 720.0
    c.facing = facing
    c.action_state = "idle"
    c.on_ground = True
    return c


def test_compute_figure_returns_geometry():
    geo = compute_figure(_standing(), _TEST_STYLE)
```

```
    assert isinstance(geo, FigureGeometry)


def test_idle_lower_foot_is_planted_at_pos_y():
    c = _standing()
    geo = compute_figure(c, _TEST_STYLE)
    lower_foot_y = max(geo.front_foot[1], geo.back_foot[1])
    assert abs(lower_foot_y - c.pos_y) < 1e-6


def test_facing_mirrors_horizontally():
    right = compute_figure(_standing(facing=1), _TEST_STYLE)
    left = compute_figure(_standing(facing=-1), _TEST_STYLE)
    # Head sits above pos_x for both; mirroring keeps it near centre but
    # flips any horizontal asymmetry of the arms.
    assert abs(right.front_hand[0] - 240) > 0  # arm extends off-centre
    assert abs((right.front_hand[0] - 240) + (left.front_hand[0] - 240)) < 1e-6
```

- [ ] **Step 2: Run the test to verify it fails**

Run: `python -m pytest pixel_battle/tests/test_poses.py::test_idle_lower_foot_is_planted_at_pos_y -v`
Expected: FAIL — `ImportError: cannot import name 'compute_figure'`

- [ ] **Step 3: Append the static poses, `FigureGeometry`, and `compute_figure`**

```
# --- Append to pixel_battle/rl/poses.py ---

# Static (non-attack) poses, authored for facing = +1.
IDLE_POSE = FigurePose(
    torso_lean=0.0,
    front_arm=(88.0, -26.0), back_arm=(100.0, -32.0),
    front_leg=(96.0, 14.0), back_leg=(84.0, 14.0),
    weapon_deg=120.0)

WALK_POSE = FigurePose(
    torso_lean=8.0,
    front_arm=(60.0, -40.0), back_arm=(130.0, -40.0),
    front_leg=(120.0, 26.0), back_leg=(60.0, 30.0),
    weapon_deg=70.0)

JUMP_POSE = FigurePose(
    torso_lean=6.0,
    front_arm=(40.0, -70.0), back_arm=(150.0, -70.0),
    front_leg=(120.0, 70.0), back_leg=(70.0, 70.0),
    weapon_deg=40.0)

HIT_POSE = FigurePose(
    torso_lean=-22.0,
```

```
    front_arm=(250.0, -60.0), back_arm=(290.0, -60.0),
    front_leg=(110.0, 50.0), back_leg=(70.0, 40.0),
    weapon_deg=300.0)


@dataclass
class FigureGeometry:
    """World-space positions for every drawable point of the figure."""
    head_center: Vec
    shoulder: Vec
    hip: Vec
    front_elbow: Vec
    front_hand: Vec
    back_elbow: Vec
    back_hand: Vec
    front_knee: Vec
    front_foot: Vec
    back_knee: Vec
    back_foot: Vec
    weapon_deg: float
    facing: int


def _mirror_abs(deg: float, facing: int) -> float:
    return deg if facing >= 0 else 180.0 - deg


def _mirror_flex(flex: float, facing: int) -> float:
    return flex if facing >= 0 else -flex


def compute_figure(char, style: dict) -> FigureGeometry:
    """Resolve `char`'s pose, run FK, and position the figure with the
    lower foot planted at `char.pos_y`."""
    pose = resolve_pose(char)
    facing = 1 if char.facing >= 0 else -1
    cx, cy = float(char.pos_x), float(char.pos_y)

    thigh, shin = style["thigh"], style["shin"]
    upper, fore = style["upper_arm"], style["forearm"]
    torso_len = style["torso_length"]

    # 1. Solve both legs from a temporary hip at the origin.
    fh_deg = _mirror_abs(pose.front_leg[0], facing)
    fk_flex = _mirror_flex(clamp_knee(pose.front_leg[1]), facing)
    bh_deg = _mirror_abs(pose.back_leg[0], facing)
    bk_flex = _mirror_flex(clamp_knee(pose.back_leg[1]), facing)
    f_knee0, f_foot0 = solve_limb((0.0, 0.0), fh_deg, fk_flex, thigh, shin)
```

```
    b_knee0, b_foot0 = solve_limb((0.0, 0.0), bh_deg, bk_flex, thigh, shin)

    # 2. Place the hip so the LOWER foot (largest y) sits at cy.
    lowest = max(f_foot0[1], b_foot0[1])
    hip = (cx, cy - lowest)

    def _shift(p):
        return (p[0] + hip[0], p[1] + hip[1])

    front_knee, front_foot = _shift(f_knee0), _shift(f_foot0)
    back_knee, back_foot = _shift(b_knee0), _shift(b_foot0)

    # 3. Torso up from hip (270 deg = straight up; + lean toward facing).
    torso_deg = _mirror_abs(270.0 + pose.torso_lean, facing)
    tc, ts = _deg2vec(torso_deg)
    shoulder = (hip[0] + tc * torso_len, hip[1] + ts * torso_len)
    head_gap = style["head_size"] + 6
    head_center = (shoulder[0] + tc * head_gap, shoulder[1] + ts * head_gap)

    # 4. Arms from the shoulder.
    fa_deg = _mirror_abs(pose.front_arm[0], facing)
    fa_flex = _mirror_flex(clamp_elbow(pose.front_arm[1]), facing)
    ba_deg = _mirror_abs(pose.back_arm[0], facing)
    ba_flex = _mirror_flex(clamp_elbow(pose.back_arm[1]), facing)
    front_elbow, front_hand = solve_limb(shoulder, fa_deg, fa_flex, upper, fore)
    back_elbow, back_hand = solve_limb(shoulder, ba_deg, ba_flex, upper, fore)

    return FigureGeometry(
        head_center=head_center, shoulder=shoulder, hip=hip,
        front_elbow=front_elbow, front_hand=front_hand,
        back_elbow=back_elbow, back_hand=back_hand,
        front_knee=front_knee, front_foot=front_foot,
        back_knee=back_knee, back_foot=back_foot,
        weapon_deg=_mirror_abs(pose.weapon_deg, facing), facing=facing)
```

- [ ] **Step 4: Append the `resolve_pose` stub for non-attack states**

`compute_figure` calls `resolve_pose`. Add it now handling the static states; Task 5 extends it for attack/kick archetypes.

```
# --- Append to pixel_battle/rl/poses.py ---

def resolve_pose(char) -> FigurePose:
    """Return the interpolated FigurePose for `char`'s current state."""
    pose_id = select_pose_id(char)
    if pose_id in ARCHETYPE_IDS:
        return _resolve_attack_pose(pose_id, char)
    if pose_id == "kick":
        return _resolve_attack_pose("kick", char)
```

```
    if pose_id == "walk":
        return WALK_POSE
    if pose_id == "jump":
        return JUMP_POSE
    if pose_id == "hit":
        return HIT_POSE
    return IDLE_POSE
```

Also add this temporary stub so `poses.py` imports cleanly until Task 5 replaces it (place it directly above `resolve_pose`):

```
def _resolve_attack_pose(pose_id: str, char) -> FigurePose:
    return IDLE_POSE   # replaced in Task 5
```

- [ ] **Step 5: Run the tests to verify they pass**

Run: `python -m pytest pixel_battle/tests/test_poses.py -v` Expected: PASS (all)

- [ ] **Step 6: Commit**

```
git add pixel_battle/rl/poses.py pixel_battle/tests/test_poses.py
git commit -m "feat(pixel-battle/rl): compute_figure + non-attack poses"
```

Task 5: The 8 archetype attack poses + kick

Files: - Modify: `pixel_battle/rl/poses.py` - Test: `pixel_battle/tests/test_poses.py`

Each attack archetype defines a `cocked` keyframe (end of windup) and an `extended` keyframe (end of strike). Phase resolution: windup = `IDLE -> cocked` (ease-in), strike = `cocked -> extended` (ease-out), recover = `extended -> IDLE` (ease-in-out).

- [ ] **Step 1: Add the failing distinctness test**

Append to `pixel_battle/tests/test_poses.py`:

```
from pixel_battle.rl.poses import ARCHETYPE_POSES
import math as _math


def _attacking(vfx, phase, phase_t, char_id="garen"):
    c = Character.load(char_id)
    c.pos_x, c.pos_y = 240.0, 720.0
    c.facing = 1
    c.action_state = "attacking"
    c.attack_anim_hint = "jab"
    c.attack_phase = phase
    c.attack_phase_t = phase_t
```

```
    class _Sk:
        pass
    s = _Sk()
    s.vfx = vfx
    c.attack_used_kind = s
    return c


def test_all_eight_archetypes_have_pose_tables():
    for a in ("melee", "slam", "spin", "dash",
              "bolt", "multishot", "aura", "beam"):
        assert a in ARCHETYPE_POSES
        assert "cocked" in ARCHETYPE_POSES[a]
        assert "extended" in ARCHETYPE_POSES[a]


def test_archetype_strike_silhouettes_are_pairwise_distinct():
    """Anti-monotony lock: each archetype's strike-end silhouette — front
    hand + front foot + back hand — must differ meaningfully from every
    other archetype's."""
    sigs = {}
    for a in ("melee", "slam", "spin", "dash",
              "bolt", "multishot", "aura", "beam"):
        geo = compute_figure(_attacking(a, "strike", 999), _TEST_STYLE)
        sigs[a] = (geo.front_hand, geo.front_foot, geo.back_hand)
    keys = list(sigs)
    for i in range(len(keys)):
        for j in range(i + 1, len(keys)):
            total = sum(_math.hypot(p[0] - q[0], p[1] - q[1])
                        for p, q in zip(sigs[keys[i]], sigs[keys[j]]))
            assert total > 25.0, \
                f"{keys[i]} vs {keys[j]} too similar ({total:.1f}px)"


def test_attack_pose_changes_across_phases():
    w = compute_figure(_attacking("melee", "windup", 10), _TEST_STYLE)
    s = compute_figure(_attacking("melee", "strike", 999), _TEST_STYLE)
    assert w.front_hand != s.front_hand
```

- [ ] **Step 2: Run the test to verify it fails**

Run: `python -m pytest pixel_battle/tests/test_poses.py::test_all_eight_archetypes_have_pose_tables -v`
Expected: FAIL — `ImportError: cannot import name 'ARCHETYPE_POSES'`

- [ ] **Step 3: Append the pose tables + real `_resolve_attack_pose`**

Replace the Task 4 stub `_resolve_attack_pose` with the real implementation, and add the data. **Delete the stub line `return IDLE_POSE # replaced in Task 5`.**

```python
# --- Append to pixel_battle/rl/poses.py ---

# Renderer-side phase durations (ms) used only to pace pose interpolation.
# They do NOT affect gameplay timing.
_PHASE_DUR: Dict[str, Dict[str, int]] = {
    "melee":     {"windup": 90,  "strike": 55,  "recover": 130},
    "slam":      {"windup": 240, "strike": 110, "recover": 260},
    "spin":      {"windup": 160, "strike": 200, "recover": 200},
    "dash":      {"windup": 130, "strike": 90,  "recover": 170},
    "bolt":      {"windup": 150, "strike": 70,  "recover": 160},
    "multishot": {"windup": 170, "strike": 110, "recover": 180},
    "aura":      {"windup": 200, "strike": 160, "recover": 220},
    "beam":      {"windup": 180, "strike": 260, "recover": 220},
    "kick":      {"windup": 120, "strike": 70,  "recover": 180},
}
_DEFAULT_PHASE_DUR = {"windup": 120, "strike": 70, "recover": 160}


def _fp(torso, fa, ba, fl, bl, wpn) -> FigurePose:
    return FigurePose(torso_lean=torso, front_arm=fa, back_arm=ba,
                      front_leg=fl, back_leg=bl, weapon_deg=wpn)


# {archetype: {"cocked": <end of windup>, "extended": <end of strike>}}
# Authored for facing = +1. Starting values — tuned in Task 11.
ARCHETYPE_POSES: Dict[str, Dict[str, FigurePose]] = {
    "melee": {
        "cocked":   _fp(-18, (215, -95), (150, -45), (110, 20), (70, 24), 250),
        "extended": _fp(24, (8, -4), (120, -30), (70, 16), (118, 22), 12),
    },
    "slam": {
        "cocked":   _fp(-30, (255, -40), (285, -40), (104, 34), (70, 30), 280),
        "extended": _fp(40, (70, -6), (95, -10), (84, 22), (96, 24), 95),
    },
    "spin": {
        "cocked":   _fp(0, (200, -10), (340, -10), (100, 18), (80, 18), 200),
        "extended": _fp(0, (20, -8), (160, -8), (110, 20), (70, 20), 20),
    },
    "dash": {
        "cocked":   _fp(-12, (210, -85), (150, -50), (70, 70), (60, 16), 240),
        "extended": _fp(46, (6, -2), (140, -36), (135, 24), (40, 8), 8),
    },
    "bolt": {
        "cocked":   _fp(-10, (140, -70), (120, -50), (104, 16), (76, 20), 150),
        "extended": _fp(16, (4, -8), (96, -40), (96, 16), (84, 18), 4),
    },
    "multishot": {
```

```
        "cocked":   _fp(-14, (188, -60), (170, -55), (108, 20), (70, 22), 200),
        "extended": _fp(20, (340, -30), (30, -30), (74, 16), (112, 20), 330),
    },
    "aura": {
        "cocked":   _fp(-8, (300, -30), (250, -30), (96, 40), (84, 40), 300),
        "extended": _fp(2, (285, -16), (255, -16), (92, 16), (88, 16), 285),
    },
    "beam": {
        "cocked":   _fp(-26, (200, -70), (170, -60), (110, 30), (60, 24), 200),
        "extended": _fp(-16, (354, -12), (6, -12), (118, 28), (52, 20), 0),
    },
    "kick": {
        "cocked":   _fp(-14, (250, -50), (290, -50), (40, 110), (88, 16), 300),
        "extended": _fp(20, (240, -40), (300, -40), (8, 6), (92, 14), 300),
    },
}


def _phase_dur(pose_id: str, phase: str) -> int:
    return _PHASE_DUR.get(pose_id, _DEFAULT_PHASE_DUR).get(
        phase, _DEFAULT_PHASE_DUR[phase])


def _resolve_attack_pose(pose_id: str, char) -> FigurePose:
    table = ARCHETYPE_POSES.get(pose_id, ARCHETYPE_POSES["melee"])
    cocked, extended = table["cocked"], table["extended"]
    phase = getattr(char, "attack_phase", "none")
    phase_t = getattr(char, "attack_phase_t", 0)
    if phase == "windup":
        t = ease_in_cubic(phase_t / _phase_dur(pose_id, "windup"))
        return lerp_pose(IDLE_POSE, cocked, t)
    if phase == "strike":
        t = ease_out_cubic(phase_t / _phase_dur(pose_id, "strike"))
        return lerp_pose(cocked, extended, t)
    if phase == "recover":
        t = ease_in_out_cubic(phase_t / _phase_dur(pose_id, "recover"))
        return lerp_pose(extended, IDLE_POSE, t)
    return cocked


def cocked_weapon_deg(pose_id: str) -> float:
    """Weapon angle at the start of the strike sweep (facing +1)."""
    table = ARCHETYPE_POSES.get(pose_id, ARCHETYPE_POSES["melee"])
    return table["cocked"].weapon_deg
```

- [ ] **Step 4: Run the tests to verify they pass**

Run: `python -m pytest pixel_battle/tests/test_poses.py -v` Expected: PASS (all). If `test_archetype_strike_silhouettes_are_pairwise_distinct` fails, nudge the offending archetype's `extended` arm/leg angles apart — the test names the two clashing archetypes.

- [ ] **Step 5: Commit**

```
git add pixel_battle/rl/poses.py pixel_battle/tests/test_poses.py
git commit -m "feat(pixel-battle/rl): 8 archetype attack poses + kick"
```

Task 6: Weapon registry + drawing

Files: - Create: `pixel_battle/rl/weapons.py` - Test: `pixel_battle/tests/test_weapons.py`

- [ ] **Step 1: Write the failing test**

```
# pixel_battle/tests/test_weapons.py
"""Weapon registry + drawing."""
import os
os.environ.setdefault("SDL_VIDEODRIVER", "dummy")

import numpy as np
import pygame
import pytest

from pixel_battle.rl.weapons import Weapon, get_weapon, draw_weapon


@pytest.fixture(scope="module", autouse=True)
def _pg():
    pygame.init()
    pygame.display.set_mode((1, 1))
    yield
    pygame.quit()


def test_champions_have_weapons():
    for cid, kind in (("garen", "greatsword"), ("lux", "staff"),
                      ("yasuo", "katana"), ("ashe", "bow")):
        w = get_weapon(cid)
        assert isinstance(w, Weapon)
        assert w.kind == kind


def test_phone_characters_are_unarmed():
    assert get_weapon("brick_phone") is None
    assert get_weapon("glass_slab") is None
    assert get_weapon("unknown_id") is None
```

```
def test_draw_weapon_marks_the_surface():
    surf = pygame.Surface((200, 200))
    surf.fill((0, 0, 0))
    w = get_weapon("garen")
    draw_weapon(surf, w, grip_xy=(100, 100), angle_deg=0.0,
                line_width=8, color=(200, 200, 200), accent=(255, 255, 255))
    arr = pygame.surfarray.array3d(surf)
    assert (arr > 0).any()
```

- [ ] **Step 2: Run the test to verify it fails**

Run: `python -m pytest pixel_battle/tests/test_weapons.py -v` Expected: FAIL — `ModuleNotFoundError: No module named 'pixel_battle.rl.weapons'`

- [ ] **Step 3: Create `weapons.py`**

```
# pixel_battle/rl/weapons.py
"""Held weapons for the renderer — registry + drawing.

Weapon appearance is renderer-side visual data, keyed by char.id, exactly
like stick_renderer._STYLES. Gameplay data stays in characters.json.
Angles use the poses.py convention (degrees, 0=right, 90=down).
"""
from __future__ import annotations
import math
from dataclasses import dataclass
from typing import Optional, Tuple

Vec = Tuple[float, float]


@dataclass
class Weapon:
    kind: str          # "greatsword" | "staff" | "katana" | "bow"
    length: float      # tip distance from the grip, px
    grip: str          # "one_hand" | "two_hand"
    width: float       # blade/shaft thickness multiplier (x line_width)


# Renderer-side registry — only the 4 LoL champions are armed.
_WEAPONS = {
    "garen":  Weapon("greatsword", length=104, grip="two_hand", width=1.7),
    "lux":    Weapon("staff",      length=110, grip="one_hand", width=0.8),
    "yasuo":  Weapon("katana",     length=92,  grip="one_hand", width=0.7),
    "ashe":   Weapon("bow",        length=84,  grip="one_hand", width=0.7),
}
```

```
def get_weapon(char_id: str) -> Optional[Weapon]:
    return _WEAPONS.get(char_id)


def _vec(deg: float) -> Vec:
    r = math.radians(deg)
    return math.cos(r), math.sin(r)


def _pt(origin: Vec, deg: float, dist: float) -> Vec:
    c, s = _vec(deg)
    return (origin[0] + c * dist, origin[1] + s * dist)


def draw_weapon(surf, weapon: Weapon, grip_xy: Vec, angle_deg: float,
                line_width: int, color, accent,
                off_hand_xy: Optional[Vec] = None) -> None:
    """Draw `weapon` gripped at `grip_xy`, pointing along `angle_deg`."""
    gx, gy = int(grip_xy[0]), int(grip_xy[1])
    tip = _pt(grip_xy, angle_deg, weapon.length)
    tip_i = (int(tip[0]), int(tip[1]))
    w = max(2, int(line_width * weapon.width))

    if weapon.kind == "greatsword":
        # Blade as a tapered quad + crossguard + stub grip.
        guard = _pt(grip_xy, angle_deg, weapon.length * 0.18)
        perp = angle_deg + 90
        half = w
        p1 = _pt(guard, perp, half)
        p2 = _pt(guard, perp, -half)
        pygame.draw.polygon(surf, accent, [p1, p2, tip_i])
        pygame.draw.polygon(surf, color, [p1, p2, tip_i], 2)
        cg1 = _pt(guard, perp, half * 2.4)
        cg2 = _pt(guard, perp, -half * 2.4)
        pygame.draw.line(surf, color, cg1, cg2, w)
        butt = _pt(grip_xy, angle_deg + 180, weapon.length * 0.16)
        pygame.draw.line(surf, color, butt, guard, w)

    elif weapon.kind == "staff":
        pygame.draw.line(surf, color, (gx, gy), tip_i, w)
        pygame.draw.circle(surf, accent, tip_i, w + 5)
        pygame.draw.circle(surf, (255, 255, 255), tip_i, w + 1)

    elif weapon.kind == "katana":
        # Slightly curved: a 3-point polyline bowed toward the back edge.
        mid = _pt(grip_xy, angle_deg, weapon.length * 0.55)
```

```
        mid = _pt(mid, angle_deg + 90, w * 1.6)
        pygame.draw.lines(surf, color, False,
                          [(gx, gy), mid, tip_i], w)
        guard = _pt(grip_xy, angle_deg, weapon.length * 0.1)
        perp = angle_deg + 90
        pygame.draw.line(surf, color, _pt(guard, perp, w * 2),
                         _pt(guard, perp, -w * 2), max(2, w - 1))

    elif weapon.kind == "bow":
        # Bow stave as an arc of points; string from tip to tip (or off-hand).
        perp = angle_deg + 90
        n = 9
        pts = []
        for i in range(n):
            f = i / (n - 1)
            along = _pt(grip_xy, angle_deg, (f - 0.5) * weapon.length)
            bow = math.sin(f * math.pi) * weapon.length * 0.22
            pts.append(_pt(along, perp, bow))
        pygame.draw.lines(surf, color, False, pts, w)
        string_anchor = off_hand_xy if off_hand_xy is not None else \
            _pt(grip_xy, angle_deg + 90, 0)
        pygame.draw.line(surf, (235, 235, 235), pts[0], string_anchor, 1)
        pygame.draw.line(surf, (235, 235, 235), pts[-1], string_anchor, 1)
```

- [ ] **Step 4: Run the tests to verify they pass**

Run: `python -m pytest pixel_battle/tests/test_weapons.py -v` Expected: PASS (3/3)

- [ ] **Step 5: Commit**

```
git add pixel_battle/rl/weapons.py pixel_battle/tests/test_weapons.py
git commit -m "feat(pixel-battle/rl): weapon registry + per-type drawing"
```

Task 7: Swing-arc smear

Files: - Modify: `pixel_battle/rl/weapons.py` - Test: `pixel_battle/tests/test_weapons.py`

- [ ] **Step 1: Add the failing test**

Append to `pixel_battle/tests/test_weapons.py`:

```
from pixel_battle.rl.weapons import draw_swing_smear


def test_swing_smear_draws_faded_copies():
    surf = pygame.Surface((300, 300))
    surf.fill((0, 0, 0))
```

```
    w = get_weapon("garen")
    draw_swing_smear(surf, w, grip_xy=(150, 150), angle_from=250.0,
                     angle_to=20.0, line_width=8, color=(200, 80, 80))
    arr = pygame.surfarray.array3d(surf)
    assert (arr > 0).any()


def test_swing_smear_noop_when_angles_equal():
    surf = pygame.Surface((300, 300))
    surf.fill((0, 0, 0))
    w = get_weapon("garen")
    draw_swing_smear(surf, w, grip_xy=(150, 150), angle_from=30.0,
                     angle_to=30.0, line_width=8, color=(200, 80, 80))
    arr = pygame.surfarray.array3d(surf)
    assert not (arr > 0).any()
```

- [ ] **Step 2: Run the test to verify it fails**

Run: `python -m pytest pixel_battle/tests/test_weapons.py::test_swing_smear_draws_faded_copies -v`
Expected: FAIL — `ImportError: cannot import name 'draw_swing_smear'`

- [ ] **Step 3: Append `draw_swing_smear` to `weapons.py`**

```
# --- Append to pixel_battle/rl/weapons.py ---

def draw_swing_smear(surf, weapon: Weapon, grip_xy: Vec,
                     angle_from: float, angle_to: float,
                     line_width: int, color) -> None:
    """Draw 3 faded weapon ghosts fanned between angle_from and angle_to,
    suggesting the blade's motion blur during a strike. No-op if the
    weapon did not sweep."""
    if abs(angle_to - angle_from) < 1.0:
        return
    w = max(2, int(line_width * weapon.width))
    sw, sh = surf.get_size()
    for i, alpha in ((0.25, 60), (0.5, 95), (0.75, 140)):
        deg = angle_from + (angle_to - angle_from) * i
        c, s = _vec(deg)
        tip = (grip_xy[0] + c * weapon.length,
               grip_xy[1] + s * weapon.length)
        ghost = pygame.Surface((sw, sh), pygame.SRCALPHA)
        pygame.draw.line(ghost, (color[0], color[1], color[2], alpha),
                         grip_xy, tip, w)
        surf.blit(ghost, (0, 0))
```

- [ ] **Step 4: Run the tests to verify they pass**

Run: `python -m pytest pixel_battle/tests/test_weapons.py -v` Expected: PASS (5/5)

- [ ] **Step 5: Commit**

```
git add pixel_battle/rl/weapons.py pixel_battle/tests/test_weapons.py
git commit -m "feat(pixel-battle/rl): weapon swing-arc motion smear"
```

Task 8: Rewire `stick_renderer.py` to the jointed skeleton

Files: - Modify: `pixel_battle/rl/stick_renderer.py` - Rewrite: `pixel_battle/tests/test_stick_renderer_pose.py`

This rewrites `draw_stick_figure` to use `compute_figure` + weapons, extends `_STYLES` with split limb lengths, and rewrites `_draw_ghost`. It **deletes** `_arm_offsets`, `_leg_offsets`, `_PHASE_DURS`, `_phase_dur`, the easing functions, and the old length constants — all superseded by `poses.py`. `spawn_impact_burst`, `spawn_landing_dust`, and `ProjectileLayer` are kept verbatim.

- [ ] **Step 1: Rewrite the renderer pose test**

Replace the entire contents of `pixel_battle/tests/test_stick_renderer_pose.py`:

```
# pixel_battle/tests/test_stick_renderer_pose.py
"""Jointed stick-figure rendering + the projectile layer."""
import os
os.environ.setdefault("SDL_VIDEODRIVER", "dummy")

import numpy as np
import pygame
import pytest

from pixel_battle.engine.character import Character
from pixel_battle.rl.stick_renderer import (
    draw_stick_figure, get_style, ProjectileLayer,
)


@pytest.fixture(scope="module", autouse=True)
def _pg():
    pygame.init()
    pygame.display.set_mode((1, 1))
    yield
    pygame.quit()


def _char(char_id, hint="jab", phase="strike", phase_t=40, vfx=None):
    c = Character.load(char_id)
    c.pos_x, c.pos_y = 240.0, 720.0
    c.facing = 1
    c.action_state = "attacking"
    c.attack_phase = phase
```

```
    c.attack_phase_t = phase_t
    c.attack_anim_hint = hint
    if vfx is not None:
        class _Sk:
            pass
        s = _Sk()
        s.vfx = vfx
        c.attack_used_kind = s
    return c


def _nonbg(surf):
    arr = pygame.surfarray.array3d(surf)
    return int(np.any(arr != 0, axis=-1).sum())


def test_every_style_has_split_limb_lengths():
    for cid in ("brick_phone", "glass_slab", "garen", "lux", "yasuo", "ashe"):
        st = get_style(cid)
        for key in ("upper_arm", "forearm", "thigh", "shin",
                    "torso_length", "head_size", "line_width"):
            assert key in st, f"{cid} style missing {key}"


def test_draw_stick_figure_marks_surface():
    surf = pygame.Surface((480, 854))
    surf.fill((0, 0, 0))
    draw_stick_figure(surf, _char("garen", vfx="melee"), (90, 205, 115))
    assert _nonbg(surf) > 200


def test_armed_character_draws_more_than_unarmed():
    """Garen (greatsword) should paint more pixels than brick_phone in the
    same pose — the weapon adds ink."""
    armed = pygame.Surface((480, 854)); armed.fill((0, 0, 0))
    unarmed = pygame.Surface((480, 854)); unarmed.fill((0, 0, 0))
    draw_stick_figure(armed, _char("garen", vfx="slam"), (90, 205, 115))
    draw_stick_figure(unarmed, _char("brick_phone", vfx="slam"), (225, 95, 80))
    assert _nonbg(armed) > _nonbg(unarmed)


def test_projectile_layer_spawn_and_decay():
    layer = ProjectileLayer()
    surf = pygame.Surface((200, 200))
    layer.spawn((10, 10), (190, 10), (255, 0, 0), current_ms=0, duration_ms=300)
    assert len(layer._items) == 1
    surf.fill((0, 0, 0))
    layer.draw(surf, 100)
```

```
    arr = pygame.surfarray.array3d(surf)
    assert (arr > 0).any()
    layer.draw(surf, 500)
    assert len(layer._items) == 0
```

- [ ] **Step 2: Run the test to verify it fails**

Run: `python -m pytest pixel_battle/tests/test_stick_renderer_pose.py -v` Expected: FAIL — `test_every_style_has_split_limb_lengths` fails (old `_STYLES` has no `upper_arm`); `ImportError` for `get_style` is also possible.

- [ ] **Step 3: Extend `_STYLES` in `stick_renderer.py`**

Replace the `_STYLES` and `_DEFAULT_STYLE` dicts (currently `stick_renderer.py:49-121`) with split limb lengths (arms ~+25% longer than the old `arm_length`). Remove the old `arm_length` / `leg_length` keys.

```
# Per-character visual style. Limb lengths are split into two segments
# (upper_arm + forearm, thigh + shin) for the jointed skeleton.
_STYLES = {
    "brick_phone": {"head_shape": "square",   "head_size": 26,
                    "torso_length": 88, "upper_arm": 30, "forearm": 30,
                    "thigh": 32, "shin": 32, "line_width": 8,
                    "hand_radius": 7, "foot_length": 22},
    "glass_slab":  {"head_shape": "triangle", "head_size": 30,
                    "torso_length": 104, "upper_arm": 31, "forearm": 31,
                    "thigh": 35, "shin": 35, "line_width": 5,
                    "hand_radius": 4, "foot_length": 14},
    "garen":       {"head_shape": "square",   "head_size": 28,
                    "torso_length": 86, "upper_arm": 32, "forearm": 31,
                    "thigh": 31, "shin": 31, "line_width": 9,
                    "hand_radius": 8, "foot_length": 22},
    "lux":         {"head_shape": "diamond",  "head_size": 30,
                    "torso_length": 108, "upper_arm": 30, "forearm": 30,
                    "thigh": 36, "shin": 36, "line_width": 5,
                    "hand_radius": 4, "foot_length": 13},
    "yasuo":       {"head_shape": "circle",   "head_size": 27,
                    "torso_length": 94, "upper_arm": 33, "forearm": 32,
                    "thigh": 33, "shin": 33, "line_width": 6,
                    "hand_radius": 5, "foot_length": 15},
    "ashe":        {"head_shape": "triangle", "head_size": 27,
                    "torso_length": 96, "upper_arm": 34, "forearm": 33,
                    "thigh": 33, "shin": 33, "line_width": 5,
                    "hand_radius": 4, "foot_length": 13},
}

_DEFAULT_STYLE = {"head_shape": "circle", "head_size": 22,
                  "torso_length": 80, "upper_arm": 28, "forearm": 28,
```

```
                    "thigh": 30, "shin": 30, "line_width": 4,
                    "hand_radius": 4, "foot_length": 12}
```

- [ ] **Step 4: Replace the renderer body**

In `stick_renderer.py`: delete the easing functions, `_lerp`, `_lerp2`, `_PHASE_DURS`, `_DEFAULT_DUR`, `_phase_dur`, `_arm_offsets`, `_leg_offsets`, and the old module-level length constants (`HEAD_RADIUS`, `TORSO_LENGTH`, `ARM_LENGTH`, `LEG_LENGTH`, `LINE_WIDTH`, `HAND_RADIUS`, `FOOT_LENGTH`, `SMEAR_VEL_THRESHOLD` is kept). Keep `get_style`, `spawn_impact_burst`, `spawn_landing_dust`, and `ProjectileLayer` exactly as they are. Replace `_draw_ghost` and `draw_stick_figure` with:

```
import pygame
from pixel_battle.rl.poses import compute_figure, cocked_weapon_deg, select_pose_id
from pixel_battle.rl.weapons import get_weapon, draw_weapon, draw_swing_smear

SMEAR_VEL_THRESHOLD = 2.5


def _draw_limb(surf, color, root, joint, end, line_width, cap_radius):
    pygame.draw.line(surf, color, root, joint, line_width)
    pygame.draw.line(surf, color, joint, end, line_width)
    pygame.draw.circle(surf, color, (int(end[0]), int(end[1])), cap_radius)


def _draw_foot(surf, color, knee, foot, line_width, foot_length):
    import math
    dx, dy = foot[0] - knee[0], foot[1] - knee[1]
    n = math.hypot(dx, dy) or 1.0
    px, py = -dy / n, dx / n
    half = foot_length / 2
    pygame.draw.line(surf, color, knee, foot, line_width)
    pygame.draw.line(surf, color,
                     (foot[0] + px * half, foot[1] + py * half),
                     (foot[0] - px * half, foot[1] - py * half), line_width)


def _draw_head(surf, color, geo, style):
    cx, cy = int(geo.head_center[0]), int(geo.head_center[1])
    hs = style["head_size"]
    shape = style["head_shape"]
    if shape == "square":
        rect = pygame.Rect(cx - hs, cy - hs, hs * 2, hs * 2)
        pygame.draw.rect(surf, color, rect)
        pygame.draw.rect(surf, (0, 0, 0), rect, 2)
    elif shape == "triangle":
        pts = [(cx, cy + hs), (cx - hs, cy - hs), (cx + hs, cy - hs)]
        pygame.draw.polygon(surf, color, pts)
        pygame.draw.polygon(surf, (0, 0, 0), pts, 2)
```

```
    elif shape == "diamond":
        pts = [(cx, cy - hs), (cx + hs, cy), (cx, cy + hs), (cx - hs, cy)]
        pygame.draw.polygon(surf, color, pts)
        pygame.draw.polygon(surf, (0, 0, 0), pts, 2)
    else:
        pygame.draw.circle(surf, color, (cx, cy), hs)
        pygame.draw.circle(surf, (0, 0, 0), (cx, cy), hs, 2)


def _draw_ghost(surf, char, color, offset_x, alpha, style):
    """Faded torso+arms ghost for fast-movement smear."""
    w, h = surf.get_size()
    ghost = pygame.Surface((w, h), pygame.SRCALPHA)
    orig_x = char.pos_x
    char.pos_x = orig_x + offset_x
    try:
        geo = compute_figure(char, style)
    finally:
        char.pos_x = orig_x
    gc = (color[0], color[1], color[2], alpha)
    lw = style["line_width"]
    pygame.draw.line(ghost, gc, geo.hip, geo.shoulder, lw)
    pygame.draw.line(ghost, gc, geo.shoulder, geo.front_elbow, lw)
    pygame.draw.line(ghost, gc, geo.front_elbow, geo.front_hand, lw)
    surf.blit(ghost, (0, 0))


def draw_stick_figure(surf, char, color):
    """Draw a jointed stick figure for `char` onto `surf` in `color`."""
    style = get_style(char.id)
    lw = style["line_width"]

    if abs(char.vel_x) > SMEAR_VEL_THRESHOLD:
        _draw_ghost(surf, char, color, -int(char.vel_x * 8), 64, style)
        _draw_ghost(surf, char, color, -int(char.vel_x * 4), 128, style)

    geo = compute_figure(char, style)

    # Back limbs first (depth).
    _draw_limb(surf, color, geo.shoulder, geo.back_elbow, geo.back_hand,
               lw, style["hand_radius"])
    _draw_foot(surf, color, geo.back_knee, geo.back_foot, lw,
               style["foot_length"])
    pygame.draw.line(surf, color, geo.hip, geo.back_knee, lw)

    # Torso + head + front leg.
    pygame.draw.line(surf, color, geo.hip, geo.shoulder, lw)
    _draw_head(surf, color, geo, style)
```

```
    _draw_foot(surf, color, geo.front_knee, geo.front_foot, lw,
               style["foot_length"])
    pygame.draw.line(surf, color, geo.hip, geo.front_knee, lw)

    # Weapon + swing smear, then the front arm grips over it.
    weapon = get_weapon(char.id)
    if weapon is not None:
        if (char.action_state == "attacking"
                and char.attack_phase == "strike"):
            pose_id = select_pose_id(char)
            ang_from = geo.weapon_deg if char.facing >= 0 \
                else 180.0 - cocked_weapon_deg(pose_id)
            draw_swing_smear(surf, weapon, geo.front_hand,
                             ang_from, geo.weapon_deg, lw, color)
        draw_weapon(surf, weapon, geo.front_hand, geo.weapon_deg, lw,
                    color, char.accent_color, off_hand_xy=geo.back_hand)

    _draw_limb(surf, color, geo.shoulder, geo.front_elbow, geo.front_hand,
               lw, style["hand_radius"])
```

Note the smear's `ang_from`: `cocked_weapon_deg` returns the facing-`+1` angle, so mirror it for `facing < 0`; `geo.weapon_deg` is already mirrored.

- [ ] **Step 5: Run the renderer tests + regression tests**

Run: `python -m pytest pixel_battle/tests/test_stick_renderer_pose.py pixel_battle/tests/test_play_richness.py pixel_battle/tests/test_skill_vfx.py -v` Expected: PASS — `draw_stick_figure` works; `play.py` HUD/VFX tests still green (the renderer signature is unchanged).

- [ ] **Step 6: Commit**

```
git add pixel_battle/rl/stick_renderer.py pixel_battle/tests/test_stick_renderer_pose.py
git commit -m "feat(pixel-battle/rl): jointed skeleton + weapons in the renderer"
```

Task 9: Visual-safety lock test

Files: - Test: `pixel_battle/tests/test_poses.py`

A regression lock: across every pose and phase the figure must keep its feet planted, its joints within range, and its bounding box on-screen.

- [ ] **Step 1: Add the failing test**

Append to `pixel_battle/tests/test_poses.py`:

```
from pixel_battle.rl.poses import (
    ELBOW_FLEX_MIN, ELBOW_FLEX_MAX, KNEE_FLEX_MIN, KNEE_FLEX_MAX,
)
```

```
# play.py camera shows CAM_VIEW_H = 502 world px with the floor framed
# ~82% down, so only ~411 px above the feet are ever on-screen.
_MAX_HALF_W = 260      # gross-error guard on horizontal splay from pos_x
_MAX_HEIGHT = 400      # figure + weapon must stay under the camera's top edge


def test_all_poses_keep_feet_planted_and_in_frame():
    from pixel_battle.rl.stick_renderer import get_style
    from pixel_battle.rl.weapons import get_weapon

    pose_specs = [("idle", "none"), ("walk", "none"),
                  ("jump", "none"), ("hit", "none")]
    for a in ("melee", "slam", "spin", "dash",
              "bolt", "multishot", "aura", "beam", "kick"):
        for ph in ("windup", "strike", "recover"):
            pose_specs.append((a, ph))

    for char_id in ("brick_phone", "glass_slab", "garen",
                    "lux", "yasuo", "ashe"):
        style = get_style(char_id)
        weapon = get_weapon(char_id)
        for pose_id, phase in pose_specs:
            for facing in (1, -1):
                c = Character.load(char_id)
                c.pos_x, c.pos_y = 240.0, 720.0
                c.facing = facing
                if phase == "none":
                    c.action_state = ("hit_stagger" if pose_id == "hit"
                                      else "idle")
                    c.on_ground = pose_id != "jump"
                    c.vel_x = 4.0 if pose_id == "walk" else 0.0
                else:
                    c.action_state = "attacking"
                    c.attack_phase = phase
                    c.attack_phase_t = 30
                    c.attack_anim_hint = ("kick" if pose_id == "kick"
                                          else "jab")

                    class _Sk:
                        pass
                    s = _Sk()
                    s.vfx = pose_id if pose_id != "kick" else "melee"
                    c.attack_used_kind = s

                geo = compute_figure(c, style)
                tag = f"{char_id}/{pose_id}/{phase}"
```

```
                # Feet: the lower foot is planted exactly at pos_y.
                lower = max(geo.front_foot[1], geo.back_foot[1])
                assert abs(lower - c.pos_y) < 1e-6, f"{tag} foot float"

                # Every drawable point — including the weapon tip — in frame.
                pts = [geo.head_center, geo.shoulder, geo.hip,
                       geo.front_elbow, geo.front_hand,
                       geo.back_elbow, geo.back_hand,
                       geo.front_knee, geo.front_foot,
                       geo.back_knee, geo.back_foot]
                if weapon is not None:
                    wr = _math.radians(geo.weapon_deg)
                    pts.append((
                        geo.front_hand[0] + _math.cos(wr) * weapon.length,
                        geo.front_hand[1] + _math.sin(wr) * weapon.length))
                for px, py in pts:
                    assert abs(px - c.pos_x) < _MAX_HALF_W, f"{tag} too wide"
                    assert c.pos_y - py < _MAX_HEIGHT, f"{tag} too tall"
                    assert py <= c.pos_y + 2, f"{tag} below ground"
```

- [ ] **Step 2: Run the test**

Run: `python -m pytest pixel_battle/tests/test_poses.py::test_all_poses_keep_feet_planted_and_in_frame -v`
Expected: PASS. If a pose is flagged too tall/wide, reduce that pose's most extreme angle in `ARCHETYPE_POSES` (Task 5) — e.g. a `slam` `cocked` arm reaching above the camera view should have its `shoulder_deg` pulled back toward the torso. Re-run until green.

- [ ] **Step 3: Commit**

```
git add pixel_battle/tests/test_poses.py
git commit -m "test(pixel-battle/rl): visual-safety lock for all poses"
```

Task 10: `play_multi` low-action curation filter

Files: - Modify: `pixel_battle/rl/play.py` (`run_one_match`) - Modify: `pixel_battle/rl/play_multi.py` - Test: `pixel_battle/tests/test_curation.py`

`run_one_match` already iterates `env.battle.events` per frame and branches on `et == "hit"`. Add a `hit` counter and return `action_score` = hits per second.

- [ ] **Step 1: Write the failing test**

```
# pixel_battle/tests/test_curation.py
"""Low-action match curation."""
import os
os.environ.setdefault("SDL_VIDEODRIVER", "dummy")
```

```python
def test_play_multi_exposes_min_action_rate():
    import pixel_battle.rl.play_multi as pm
    assert hasattr(pm, "MIN_ACTION_RATE")
    assert pm.MIN_ACTION_RATE > 0


def test_should_keep_match_logic():
    """A KO match below the action-rate floor is dropped; a brisk one is kept."""
    from pixel_battle.rl.play_multi import _should_keep

    assert _should_keep({"finished_by_ko": True, "action_score": 1.5})
    assert not _should_keep({"finished_by_ko": True, "action_score": 0.2})
    assert not _should_keep({"finished_by_ko": False, "action_score": 9.0})


def test_run_one_match_result_has_action_score():
    """run_one_match's result dict carries a numeric action_score."""
    import inspect
    from pixel_battle.rl import play
    src = inspect.getsource(play.run_one_match)
    assert "action_score" in src
```

- [ ] **Step 2: Run the test to verify it fails**

Run: `python -m pytest pixel_battle/tests/test_curation.py -v` Expected: FAIL — `MIN_ACTION_RATE` / `_should_keep` do not exist yet.

- [ ] **Step 3: Add the `hit` counter + `action_score` to `run_one_match`**

In `pixel_battle/rl/play.py`, inside `run_one_match`:

1. Before the per-frame loop, initialise the counter alongside the other accumulators (near `event_video_ms: dict = {}`, ~line 401):

   ```python
   hit_count = 0
   ```

2. In the event loop where `et == "hit"` is already handled (~line 437), add one line inside that branch:

   ```python
   if et == "hit":
       hit_count += 1
       # ... existing hit handling unchanged ...
   ```

3. Where the result dict is built (the `return {...}` carrying `finished_by_ko`, `winner`, `duration_s`, `mp4_path`, `raw_path`, `audio_path`), add:

   ```python
   "action_score": hit_count / max(duration_s, 0.1),
   ```

Use the same `duration_s` value the dict already returns.

- [ ] **Step 4: Add the filter to `play_multi.py`**

In `pixel_battle/rl/play_multi.py`:

1. Add the constant near `OUT_DIR` (~line 20):

```
# Drop KO matches below this hits-per-second floor — they read as
# low-action (fighters circling/retreating). Tuned in Task 11.
MIN_ACTION_RATE = 0.8
```

2. Add the predicate above `main`:

```
def _should_keep(result: dict) -> bool:
    """Keep a match only if it ended in KO and was action-dense."""
    return (result.get("finished_by_ko", False)
            and result.get("action_score", 0.0) >= MIN_ACTION_RATE)
```

3. In `main`, replace the acceptance check `if result["finished_by_ko"]:` with `if _should_keep(result):`. In the `else` branch, change the skip message so it reports the reason:

```
reason = "no KO" if not result["finished_by_ko"] else \
    f"low action {result['action_score']:.2f}/s"
print(f"⏭️  Skipped seed={seed} ({reason})")
```

- [ ] **Step 5: Run the tests + the play_multi import regression**

Run: `python -m pytest pixel_battle/tests/test_curation.py pixel_battle/tests/test_play_multi_imports.py -v` Expected: PASS.

- [ ] **Step 6: Commit**

```
git add pixel_battle/rl/play.py pixel_battle/rl/play_multi.py pixel_battle/tests/test_curation.py
git commit -m "feat(pixel-battle/rl): drop low-action matches in play_multi"
```

Task 11: Full test sweep + validation render + tuning

Files: - Possibly modify: `pixel_battle/rl/poses.py` (pose-angle tuning), `pixel_battle/rl/play_multi.py` (`MIN_ACTION_RATE` tuning)

- [ ] **Step 1: Run the entire test suite**

Run: `python -m pytest pixel_battle/tests/ -v` Expected: all green. Fix any regression before continuing.

- [ ] **Step 2: Render a single validation episode**

Run: `python -m pixel_battle.rl.play` Expected: `pixel_battle/output/rl_play/episode.mp4` is regenerated without error.

- [ ] **Step 3: Render a multi-match batch (exercises curation)**

Run: `python -m pixel_battle.rl.play_multi --num_matches 5` Expected: up to 5 `match_*.mp4` files in `pixel_battle/output/rl_play_multi/`; the console shows any seeds skipped for `low action`.

- [ ] **Step 4: Watch the output and tune**

Open `pixel_battle/output/rl_play/episode.mp4`. Check against the spec's success criteria: motions read big and weighty, the 8 archetypes look distinct, weapons read clearly (Garen greatsword, Lux staff, Yasuo katana, Ashe bow), no broken frames (clipped limbs, floating feet, off-screen weapons).

Tune as needed and re-render: - Pose too small / too large → adjust that archetype's `cocked` / `extended` angles in `ARCHETYPE_POSES` (`poses.py`). - A limb clips off-screen → the visual-safety test should already catch it; if it slips through, tighten `_MAX_HALF_W` / `_MAX_HEIGHT` and the pose. - Curation too aggressive / too lax → adjust `MIN_ACTION_RATE` in `play_multi.py`. After each tuning change, re-run `python -m pytest pixel_battle/tests/ -v`.

- [ ] **Step 5: Commit any tuning changes**

```
git add pixel_battle/rl/poses.py pixel_battle/rl/play_multi.py
git commit -m "tune(pixel-battle/rl): combat-animation pose + curation tuning"
```

Self-Review

Spec coverage — every spec section maps to a task: - §5.1 skeleton model → Tasks 1, 4 - §5.2 file layout → Tasks 1–10 (`poses.py` , `weapons.py` , `stick_renderer.py` , `play.py` , `play_multi.py`) - §5.3 data flow → Task 8 `draw_stick_figure` - §6 the 8 archetype poses + non-attack poses → Tasks 4, 5 - §7 weapons → Task 6 - §8 swing-arc smear → Task 7 - §9 visual-safety guarantees → Task 9 - §10 curation filter → Task 10 - §11 error handling → Task 3 (unknown vfx → melee), Task 5 (`ARCHETYPE_POSES.get(... , melee)`), Task 6 (`get_weapon` → `None`), Task 1 (clamps) - §12 testing → every task is TDD; Task 8 rewrites the superseded `test_stick_renderer_pose.py` ; Task 11 runs the full sweep - §13 validation → Task 11

Deviation from spec §12: the spec said `test_stick_renderer_pose.py` "stays green". In fact its `_arm_offsets` / `_leg_offsets` tests test functions that this work deletes, so Task 8 rewrites that file (keeping the `ProjectileLayer` test). `test_play_richness.py` and `test_skill_vfx.py` are genuinely unaffected and stay green.

Type consistency — checked: `FigurePose` , `FigureGeometry` , `Weapon` field names; `solve_limb` / `compute_figure` / `select_pose_id` / `resolve_pose` / `cocked_weapon_deg` signatures; `ARCHETYPE_POSES` / `ARCHETYPE_IDS` / `_PHASE_DUR` names; `get_weapon` / `draw_weapon` / `draw_swing_smear` signatures; `_STYLES` keys (`upper_arm` , `forearm` , `thigh` , `shin` , `torso_length` , `head_size` , `head_shape` , `line_width` , `hand_radius` , `foot_length`) match between Task 8's `_STYLES` and `compute_figure` 's usage in Task 4.

No placeholders — all pose-angle numbers are concrete starting values; Task 11 is the explicit tuning pass. `MIN_ACTION_RATE = 0.8` is a concrete tunable constant.